

Projet ASD : Structures de Données Hybrides

Seydina Mohamed Diop & Abdou Lahate

13 juin 2026

1 Introduction

Ce Projet porte sur un systeme de gestion de scolarité . En effet on va concevoir un programme qui va permettre a un utilisateur d enregistrer autant d etudiant qu il souhaite accompagner de leur information personnel et pedagogique .Ainsi une serie de message vont s afficher sur le terminal permettant a l utilisateur d echanger avec la machine .Ce dernier fera des series de calculs liés aux **Structures** et **Fonctions** ci dessous et affichera les informations correspondantes.

2 Modélisation des Structures de Données

Voici les Structures de base que nous avons conçues pour notre **Systeme De Scolarité**.

```
1 typedef struct Note
2 {
3     char matier[20]; // ici on donne la matiere
4     float valeur; // sa valeur
5     int coef;
6     struct Note *note_suivante; // permet de passer au note
        suivant du meme etudiant
7 }Note;
```

2.0.1 Explication de la structure Note

La structure **Note** reçoit et stocke les résultats académiques d'un étudiant pour chaque matière de la filière, ainsi que le coefficient correspondant. Par exemple, pour un étudiant donné, cette structure va enregistrer ses notes au fur et à mesure. Ensuite, grâce au pointeur `struct Note *note_suivante`, la structure se connecte à la note suivante du même étudiant. C'est ce mécanisme

qui forme une liste chaînée de notes propre à chaque étudiant, permettant de stocker un nombre dynamique de modules sans utiliser de tableau statique.

```

1 typedef struct Etudiant
2 {
3     char nom[50];
4     char prenom[50];
5     int numero_dossier; // identifiant unique par etudiant
6     bool eta_inscription; // 1 si l etudiant s'est inscrit et
        0 sinon (initialise lors de la creation)
7     float moyenne_general;
8     Note *modules; // est un pointeur de type Note
9     struct Etudiant *etudiant_suivant; // permet de passer a
        l etudiant suivant
10 }Etudiant;

```

2.0.2 Explication de la structure Etudiant

La structure **Etudiant** regroupe l'ensemble des informations d'un étudiant de la filière, telles que son nom, son prénom, son numéro de dossier (identifiant unique) et son état d'inscription (via un booleen). Le pointeur **modules** est essentielle : il pointe vers la première note de l'étudiant, liant ainsi l'étudiant à ses résultats. Concernant le pointeur **etudiant_suivant** elle pointe sur l etudiant suivante creent ainsi une liste chaine

```

1 typedef struct
2 {
3     char nom_filiere[60];
4     int nb_etudiants;
5     Etudiant *premier_du_liste; // point sur le premier de la
        liste des etudiant
6     Etudiant *dernier_du_liste; // point sur le dernier
        etudiant de la liste
7 }Filiere;

```

2.0.3 Explication de la structure Filiere

Cette structure contient le nom d'une filiere donner ainsi que le nombre d'étudiants. Dans la structure **Filiere** on a deux pointeurs : **premier_du_liste** qui pointe sur le premier étudiant de la liste du filière, **dernier_du_liste** quant a lui pointe sur le dernier étudiant. Permettant d inserer des etudiants directemant a la fin de la liste

3 Modélisation des Fonctions

Voici les fonctions que nous avons conçue pour permettre le bon fonctionnement de notre Systeme de scolarité.

```
1 Etudiant *inscrire_etudiants(Filiere *filiere)
2 {
3     // allocation dynamique d un nouveau etudiant
4     Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(
        Etudiant));
5     if (nouveau_etudiant == NULL)
6     {
7         printf("Ereure: impossible d allouer la memoire");
8         return;
9     }
10    // ajout des information d un etudiant par l utilisateur
11    printf("\n-----Formulaire d inscription-----\n");
12    printf("Donner le nom de l etudiant: \n");
13    scanf("%s", nouveau_etudiant->nom);
14    printf("Donner le prenom de l etudiant: \n");
15    scanf("%s", nouveau_etudiant->prenom);
16    printf("Donner son numero de dossier: \n");
17    scanf("%d", &nouveau_etudiant->numero_dossier);
18
19    nouveau_etudiant->eta_inscription = 1;    // on va
        enregistrer l etat d inscription a 1 par default
20    nouveau_etudiant->modules = NULL ; // pas encore de note
21    nouveau_etudiant->etudiant_suivant = NULL; // se sera le
        dernier etudiant
22
23    if (filiere->premier_du_liste == NULL)
24    {
25        // La liste est vide, il est le tout premier et le
        tout dernier
26        filiere->premier_du_liste = nouveau_etudiant;
27        filiere->dernier_du_liste = nouveau_etudiant;
28    } else
29    {
30        filiere->dernier_du_liste->etudiant_suivant =
            nouveau_etudiant; // le pointeur dernier_du_liste
            pointe sure le nouveau etudiant
31        filiere->dernier_du_liste = nouveau_etudiant; // Il
            devient le nouveau dernier de la filiere
32    }
33
34    filiere->nb_etudiants++; // On incremente le compteur de
        la classe
35    printf("L'etudiant %s %s a ete inscrit avec succes!\n",
        nouveau_etudiant->prenom, nouveau_etudiant->nom);
```

```

36     return nouveau_etudiant;
37 }

```

3.0.1 Explication de la Fonctions inscrire_etudiant

Cette fonction nous permet d'ajouter de nouveaux etudiant dans une filiere en enregistrant leur information personnel : prenom, nom, numero de dossier grace a l'allocation dynamique suivant : `Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(Etudiant))` qui alloue une espace memoire dans le memoire RAM. Combiner au boucle suivant.

```

1     if (filiere->premier_du_liste == NULL)
2     {
3         // La liste est vide, il est le tout premier et le
           tout dernier
4         filiere->premier_du_liste = nouveau_etudiant;
5         filiere->dernier_du_liste = nouveau_etudiant;
6     } else
7     {
8         filiere->dernier_du_liste->etudiant_suivant =
           nouveau_etudiant; // le pointeur dernier_du_liste
           pointe sure le nouveau etudiant
9         filiere->dernier_du_liste = nouveau_etudiant; // Il
           devient le nouveau dernier de la filiere
10    }

```

L'utilisateur pourra ajouter autant d'etudiant qu'il souhaite, le programme aura ainsi le nombre totale d'etudiants.

```

1 void note_etudiants(Etudiant *etudiant) // une fonction
   permettant d'enregistrer les resultat academique d'un
   etudiant
2 {
3     if (etudiant == NULL) return;
4
5     // On verifie si l'etudiant est bien inscrit
6     if (etudiant->eta_inscription == 0)
7     {
8         printf("Impossible d'ajouter une note : cet etudiant
           n'est pas activement inscrit.\n");
9         return;
10    }
11    int nouveaux_matiere = 2;
12    while (nouveaux_matiere == 2)
13    {
14        Note *nouveau_note = (Note*)malloc(sizeof(Note)); //
           allocation dynamique
15        if (nouveau_note == NULL)

```

```

16     {
17     printf("Ereure : impossible d'allouer la memoire");
18     return;
19     }
20
21
22     // ajout du resultat d un etudiant
23     printf("\n-----Resultats Academique-----\n");
24     printf("Donner la matiere : \n");
25     scanf("%s", nouveau_note->matier);
26     do
27     {
28         printf("Donner la note obtenue : \n");
29         scanf("%f", &nouveau_note->valeur);
30         if (nouveau_note->valeur < 0 || nouveau_note->
            valeur > 20)
31         {
32             printf("Erreur ! La note doit etre comprise
                entre 0 et 20.\n");
33         }
34     } while (nouveau_note->valeur < 0 || nouveau_note->
        valeur > 20);
35     do
36     {
37         printf("Donner le coefficient du matiere : \n");
38         scanf("%d", &nouveau_note->coef);
39         if (nouveau_note->coef <= 0 || nouveau_note->coef
            > 10)
40         {
41             printf("Erreur ! Le coefficient doit etre
                compris entre 1 et 10.\n");
42         }
43     } while (nouveau_note->coef <= 0 || nouveau_note->
        coef > 10);
44
45     nouveau_note->note_suivante = NULL; // c est la
        premiere et dernier note
46
47     if (etudiant->modules == NULL)
48     {
49         // Cas ou l etudiant n a encore aucune note
50         etudiant->modules = nouveau_note;
51     }
52     else
53     {
54         // Cas ou l'etudiant a deja des notes , on
            parcourt pour aller a la fin
55         Note *note_restante = etudiant->modules;
56         while (note_restante->note_suivante != NULL)

```

```

57         {
58             note_restante = note_restante->note_suivante;
59         }
60         // On accroche la nouvelle note tout a la fin
61         note_restante->note_suivante = nouveau_note;
62     }
63     printf("Matiere ajouter avec succe:\n");
64     printf("Si vous voulez ajouter une nouvelle matiere \n
        tape 2 sinon 0:\n");
65     scanf("%d", &nouveaux_matiere);
66 }
67 }

```

3.0.2 Explication de la Fonctions note_etudiant

Comme la fonction precedente celui ci Permet de stocke les resultats scolaire de chaque etudiants enregistrer en amont ,grace a l allocation dynamique suivant `Note *nouveau_note = (Note*)malloc(sizeof(Note))`. Cette fonction va permettre a l utilisateur de donner autant de matiere ainsi que les note et coefficient associer. Pour chacune des etudiants qu il enregistre il peut aussi ne pas ajouter de note pour les etudiants qu il souhaite . Et cela grace au boucle `while (nouveaux_matiere == 2)` qui lui dira de taper 2 s il veut ajouter des matiere et 0 sinon .

```

1 void liste_de_la_classe(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4         // verifie si la filiere est vide ou non
5     {
6         printf("Aucun etudiant inscrit dans la filiere.\n");
7         return;
8     }
9     printf("\n-----Liste des etudiants de la classe
        -----\n");
10    Etudiant *actuel_liste = filiere->premier_du_liste;
11    while (actuel_liste != NULL) // affiche les identifiant
        des etudiants
12    {
13        printf("Nom: %s, Prenom: %s, Numero de dossier: %d\n"
            , actuel_liste->nom, actuel_liste->prenom,
            actuel_liste->numero_dossier);
14        actuel_liste = actuel_liste->etudiant_suivant;
15    }
16 }

```

3.0.3 Explication de la Fonctions liste_de_la_classe

Quant a cette fonction il affichera la liste des etudiants que l utilisateur a deja enregistrer .Il donne respectivement leur nom, prenom et numero de dossier.

```
1 void bulletin(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Aucun_étudiant_inscrit_dans_la_filiere.\n");
6         return;
7     }
8
9     printf("\n-----Bulletin_des_étudiants_de_la_classe
10          -----\n");
11
12     // VARIABLES AJOUTEES POUR LA MOYENNE GENERALE (Ligne 1
13     // et 2)
14     float somme_des_moyennes_classe = 0;
15     int etudiants_avec_notes = 0; // prend en compte que les
16     // etudiants ayant une moyenne
17
18     Etudiant *actuel_liste_info = filiere->premier_du_liste;
19
20     while (actuel_liste_info != NULL)
21     {
22         // On verifie le statut de chaque etudiant
23         // individuellement ici
24         if (actuel_liste_info->eta_inscription == 1)
25         {
26             printf("Nom:_%s,_Prenom:_%s,_Numero_de_dossier:_%d\n",
27                    actuel_liste_info->nom,
28                    actuel_liste_info->prenom, actuel_liste_info->
29                    numero_dossier);
30             Note *actuel_liste_bulletin = actuel_liste_info->
31             modules;
32             float point_totale = 0;
33             int coef_totale = 0;
34
35             while (actuel_liste_bulletin != NULL)
36             {
37                 printf("Matiere:_%s,_Note:_%f,_Coeficiant:_%d\n",
38                        actuel_liste_bulletin->matier,
39                        actuel_liste_bulletin->valeur,
40                        actuel_liste_bulletin->coef);
41                 point_totale += actuel_liste_bulletin->valeur
42                     * actuel_liste_bulletin->coef;
43                 coef_totale += actuel_liste_bulletin->coef;
44             }
45         }
46         actuel_liste_info = actuel_liste_info->suivant;
47     }
48     printf("\nMoyenne generale de la classe: %f\n",
49            somme_des_moyennes_classe / etudiants_avec_notes);
50 }
```

```

32         actuel_liste_bulletin = actuel_liste_bulletin
           ->note_suivante;
33     }
34
35     if (coef_totale != 0)
36     {
37         actuel_liste_info->moyenne_general =
           point_totale / coef_totale;
38         printf("Moyenne_Generale: %.2f\n\n",
           actuel_liste_info->moyenne_general);
39         somme_des_moyennes_classe +=
           actuel_liste_info->moyenne_general;
40         etudiants_avec_notes++;
41     }
42     else
43     {
44         actuel_liste_info->moyenne_general = 0;
45         printf("Aucune_note_enregistre\n");
46     }
47 }
48 // On passe l' etudiant suivant
49 actuel_liste_info = actuel_liste_info->
   etudiant_suivant;
50 }
51
52 // Affichage de la moyenne gennerale
53 if (etudiants_avec_notes != 0)
54 {
55     float moyenne_general_classe =
           somme_des_moyennes_classe / etudiants_avec_notes;
56     printf("MOYENNE_GENERALE_DE_LA_FILIERE: %.2f / 20\n"
           , moyenne_general_classe);
57 }
58 else
59 {
60     printf("Les etudiants non pas de note.\n");
61 }
62 }

```

3.0.4 Explication de la Fonctions bulletin

La fonction bulletin permet de calculer les moyenne de chaque etudiant ainsi que la moyenne gennerale de la classe. Elle va aller chercher les module de chaque etudiants c est a dire ces matiere ainsi que les coef et notes associer pour calculer sa moyenne. Si l etudiant n a pas de note la boucle :

```

1         else
2         {

```



```

3         actuel_liste_info->moyenne_general = 0;
4         printf("Aucune note enregistre\n");
5     }

```

Va afficher a l ecran que cette etudiant n a pas de note ,cette etudiant ne sera pas prise en compte pour le calcul de la moyenne gennerale de la classe . Grace au variable `etudiants_avec_notes` , innicialise a 0 qui n avance que si l etudiant qu on doit calculer la moyenne a une note .De cette facon avec la boucle :

```

1     if (etudiants_avec_notes != 0)
2     {
3         float moyenne_general_classe =
4             somme_des_moyennes_classe / etudiants_avec_notes;
5         printf("MOYENNE GENERALE DE LA FILIERE : %.2f / 20\n",
6             , moyenne_general_classe);
7     }
8     else
9     {
10        printf("Les etudiants non pas de note.\n");
11    }
12 }

```

Cqlcule et affiche la moyenne gennerale exacte de la classe.

```

1 void recherche_et_modification_etudiant(Filiere *filiere)
2 {
3     int votre_choix = 3;
4     while (votre_choix == 3)
5     {
6         printf("Voulez vous rechercher ou modifier les info d
7             un autre etudiant si oui taper 3 si non 0:\n");
8         scanf("%d", &votre_choix);
9         if (votre_choix == 0)
10        {
11            break;
12        }
13        if (filiere == NULL || filiere->premier_du_liste ==
14            NULL)
15        {
16            printf("Aucun etudiant dans la filiere.\n");
17            return;
18        }
19        int id_rechercher;
20        printf("Donner le numero de dossier de l etudiant
21            rechercher:\n");
22        scanf("%d", &id_rechercher);
23        Etudiant *etudiant_actuel = filiere->premier_du_liste
24            ;

```

```

22     while (etudiant_actuel != NULL && etudiant_actuel->
23             numero_dossier != id_rechercher)
24     {
25         etudiant_actuel = etudiant_actuel->
26             etudiant_suivant;
27     }
28     if (etudiant_actuel == NULL)
29     {
30         printf("%d ne correspond a aucun etudiant
31             enregistre:\n", id_rechercher);
32     }
33     else
34     {
35         int choix;
36         printf("\n-----Etudiant trouver-----\n");
37         printf("Nom: %s, Prenom: %s, Numero de dossier: %d,
38             Etat d'inscription: %s", etudiant_actuel->
39             nom, etudiant_actuel->prenom, etudiant_actuel
40             ->numero_dossier, (etudiant_actuel->
41             eta_inscription == 1) ? "Inscrit" : "Non
42             inscrit");
43         printf("\n-----Que souhaitez-vous modifier
44             -----\n");
45         printf("1 Le Nom\n");
46         printf("2 Le Prenom\n");
47         printf("3 Le Numero de dossier\n");
48         printf("4 L'etat d'inscription (1: inscrit, 0:
49             Suspendu)\n");
50         printf("5 Matiere Note et Coef\n");
51         printf("Votre choix: ");
52         scanf("%d", &choix);
53
54         switch (choix) // cette partie va permettre a l
55             utilisateur de modifier les parametre qu il
56             souhaite
57         {
58             case 1:
59                 printf("Entrez le nouveau nom:\n");
60                 scanf("%s", etudiant_actuel->nom);
61                 printf("Nom modifie avec succes!\n");
62                 break;
63             case 2:
64                 printf("Entrez le nouveau prenom:\n");
65                 scanf("%s", etudiant_actuel->prenom);
66                 printf("Prenom modifie avec succes!\n");
67                 break;
68             case 3:
69                 printf("Entrez le nouveau numero de
70                     dossier:\n");

```

```

58         scanf("%d", &etudiant_actuel->
59             numero_dossier);
60         printf("Numero_de_dossier_modifie_avec_
61             succes!\n");
62         break;
63     case 4:
64         printf("Entrez_le_nouveau_statut_(1_pour_
65             Inscrit,_0_pour_Non_inscrit):\n");
66         scanf("%d", &etudiant_actuel->
67             eta_inscription);
68         printf("Statut_d'inscription_mis_a_jour_
69             !\n");
70     case 5:
71         printf("Entrez_la_nouvelle_matiere:\n");
72         scanf("%s", &etudiant_actuel->modules->
73             matier);
74         printf("Entrez_la_nouvelle_note:\n");
75         scanf("%f", &etudiant_actuel->modules->
76             valeur);
77         printf("Entrez_le_nouveau_coef:\n");
78         scanf("%d", &etudiant_actuel->modules->
79             coef);
80         printf("La_matiere_ainsi_que_sa_note_et_
81             son_coef_on_ete_modifie_avec_succes!\n");
82         break;
83     default:
84         printf("Choix_invalide._Aucune_
85             modification_effectuee.\n");
86         break;
87 }
88 }
89 }
90 }

```

3.0.5 Explication de la Fonctions recherche_et_modification_etudiant

Cette fonction va demander a l'utilisateur s'il veut rechercher un étudiant afin de modifier ou non ces informations personnelles ou académiques en lui affichant un message lui permettant de faire deux choix soit taper 3 pour continuer et avoir la possibilité de modifier certaines informations. Soit taper 0 pour arrêter le programme, et éventuellement ne rien modifier ou sortir après des modifications.

```

1 void supprimer_etudiant(Filiere *filiere)
2 {

```

```

3     int votre_choix = 4;
4     while (votre_choix == 4)
5     {
6         printf("Si vous voulez supprimer une autre etudiant
           tape 4 sinon 0:\n");
7         scanf("%d", &votre_choix);
8         if (votre_choix == 0)
9         {
10            break;
11        }
12
13        if (filiere == NULL || filiere->premier_du_liste ==
           NULL)
14        {
15            printf("Il n'y a aucune etudiant");
16            return;
17        }
18        int id_supprimer;
19        printf("Donner le numero de dossier de l etudiant que
           vous souhaitez supprimer de la filiere:\n");
20        scanf("%d", &id_supprimer);
21        Etudiant *actuel_etudiant = filiere->premier_du_liste
           ;
22        Etudiant *precedent_etudiant = NULL;
23        while (actuel_etudiant != NULL && id_supprimer !=
           actuel_etudiant->numero_dossier)
24        {
25            precedent_etudiant = actuel_etudiant; // 1
           etudiant precedent devient l actuel etudiant
26            actuel_etudiant = actuel_etudiant->
           etudiant_suivant; // 1 actuel etudiant pas a l
           etudiant suivant
27        }
28        if (actuel_etudiant == NULL)
29        {
30            printf("Etudiant introuvable avec l id: %d\n",
           id_supprimer);
31        }
32        else
33        {
34            if (precedent_etudiant == NULL)
35            {
36                // correspond au premier etudiant
37                filiere->premier_du_liste = actuel_etudiant->
           etudiant_suivant;
38                // cas ou la liste contient q un seul etudiant
39                if (filiere->premier_du_liste == NULL)
40                {
41                    filiere->dernier_du_liste = NULL;

```

```

42         }
43
44     }
45     else
46     {
47         // 1 etudiant est au milieu ou a la fin
48         precedent_etudiant->etudiant_suivant =
49             actuel_etudiant->etudiant_suivant;
50         // Si on a supprimé le dernier, l'avant-
51         // dernier (precedent) devient le nouveau
52         // dernier
53         if (actuel_etudiant == filiere->
54             dernier_du_liste)
55         {
56             filiere->dernier_du_liste =
57                 precedent_etudiant;
58         }
59     }
60 }

```

3.0.6 Explication de la Fonctions supprimer_etudiant

De la meme maniere que la fonction precedente celui ci lui permettra de supprimer ou non un etudiant de la liste

```

1 void maximum(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune etudiant");
6         return;
7     }
8     float majorant = 0.0;
9     Etudiant *etudiant_major = filiere->premier_du_liste;
10    Etudiant *le_major = NULL; // Pointeur pour memoriser le
11    // majorant
12    while (etudiant_major != NULL)
13    {
14        if (etudiant_major->eta_inscription == 1 &&
15            etudiant_major->moyenne_general >= 0.0)
16        {

```

```

15         if (etudiant_major->moyenne_general > majorant)
16         {
17             majorant = etudiant_major->moyenne_general;
18             // mise a jour du nouveaux majorant
19             le_major = etudiant_major; // memorise l
20             enplacement du majorant actuelle
21         }
22     }
23     etudiant_major = etudiant_major->etudiant_suivant;
24 }
25 if (le_major != NULL)
26 {
27     printf("Le_majorant_de_la_filieres:%s,est%s,%s,
28     numero_de_dossier:%d,et_a_une_moyenne_generale_
29     de:%.2f\n",filieres->nom_filieres, le_major->prenom
30     , le_major->nom, le_major->numero_dossier,
31     le_major->moyenne_general);
32 }
33 }

```

3.0.7 Explication de la Fonctions maximum

Cette fonction va chercher parmit tout les etudiant celui qui a la plus grande moyenne et a la fin du programme il donne sont nom prenom numero de dossier sa moyenne et le nom de la filiere.

```

1 void minimum(Filieres *filieres)
2 {
3     if (filieres == NULL || filieres->premier_du_liste == NULL)
4     {
5         printf("Il_n_y_a_aucune_etudiant");
6         return;
7     }
8     float minorant = 20;
9     Etudiant *etudiant_minor = filieres->premier_du_liste;
10    Etudiant *le_minor = NULL; // Pointeur pour m moriser le
11    minorant
12    while (etudiant_minor != NULL)
13    {
14        if (etudiant_minor->eta_inscription == 1 &&
15            etudiant_minor->moyenne_general >= 0.0)
16        {
17            if (etudiant_minor->moyenne_general < minorant)
18            {
19                minorant = etudiant_minor->moyenne_general;
20                // mise a jour du nouveaux minorant
21                le_minor = etudiant_minor; // memorise l
22                enplacement du minorant actuelle

```

```

19         }
20     }
21     etudiant_minor = etudiant_minor->etudiant_suivant;
22 }
23 if (le_minor != NULL)
24 {
25     printf("L'etudiant ayant la plus faible note de la
        filiere:%s, est %s, %s, numero de dossier: %d,
        et a une moyenne generale de: %.2f\n", filiere->
        nom_filiere, le_minor->prenom, le_minor->nom,
        le_minor->numero_dossier, le_minor->
        moyenne_general);
26 }
27 }

```

3.0.8 Explication de la Fonctions minimum

Pareil pour la fonction minimum ,il donne l etudiant ayant la plus faible moyenne de la filiere .

```

1 void du_majorant_minorant(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune etudiant");
6         return;
7     }
8     Etudiant *i = filiere->premier_du_liste;
9     Etudiant *j;
10    Etudiant *major_minor = NULL; // Pointeur pour majoriser
        le i
11    // il nous faut des variable et une pointeur pour les
        echange fineaux
12    char tmp_nom[50];
13    char tmp_prenom[50];
14    int tmp_id;
15    int tmp_etat_inscription;
16    float tmp_moyenne_general;
17    Note *tmp_modules;
18    while (i != NULL && i->etudiant_suivant != NULL)
19    {
20        major_minor = i; // on suppose que le plus grant est
        le i
21        j = i->etudiant_suivant;
22        while (j != NULL)
23        {
24            if (j->moyenne_general > major_minor->
                moyenne_general)

```

```

25         {
26             major_minor = j; // j devient temporairement
                le plus grand
27         }
28         j = j->etudiant_suivant; // j passe a l etudiant
                suivant
29     }
30     if (major_minor != i)
31     {
32         strcpy(tmp_nom, i->nom);
33         strcpy(tmp_prenom, i->prenom);
34         tmp_id = i->numero_dossier;
35         tmp_etat_inscription = i->eta_inscription;
36         tmp_moyenne_general = i->moyenne_general;
37         tmp_modules = i->modules;
38
39         strcpy(i->nom, major_minor->nom);
40         strcpy(i->prenom, major_minor->prenom);
41         i->numero_dossier = major_minor->numero_dossier;
42         i->eta_inscription = major_minor->eta_inscription
            ;
43         i->moyenne_general = major_minor->moyenne_general
            ;
44         i->modules = major_minor->modules;
45
46         strcpy(major_minor->nom, tmp_nom);
47         strcpy(major_minor->prenom, tmp_prenom);
48         major_minor->numero_dossier = tmp_id;
49         major_minor->eta_inscription =
            tmp_etat_inscription;
50         major_minor->moyenne_general =
            tmp_moyenne_general;
51         major_minor->modules = tmp_modules;
52     }
53     i = i->etudiant_suivant;
54 }
55 }

```

3.0.9 Explication de la Fonctions du majorant_minorant

Cette fonction Permet d afficher par ordre de merite tout les etudiant de la filiere du majorant a l etudiant ayant la plus faible moyenne grace a la trie par selection :

```

1     while (i != NULL && i->etudiant_suivant != NULL)
2     {
3         major_minor = i; // on suppose que le plus grant est
                le i

```



```

4      j = i->etudiant_suivant;
5      while (j != NULL)
6      {
7          if (j->moyenne_general > major_minor->
            moyenne_general)
8          {
9              major_minor = j; // j devient temporairement
                le plus grand
10         }
11         j = j->etudiant_suivant; // j passe a l etudiant
                suivant
12     }
13     if (major_minor != i)
14     {
15         strcpy(tmp_nom, i->nom);
16         strcpy(tmp_prenom, i->prenom);
17         tmp_id = i->numero_dossier;
18         tmp_etat_inscription = i->eta_inscription;
19         tmp_moyenne_general = i->moyenne_general;
20         tmp_modules = i->modules;
21
22         strcpy(i->nom, major_minor->nom);
23         strcpy(i->prenom, major_minor->prenom);
24         i->numero_dossier = major_minor->numero_dossier;
25         i->eta_inscription = major_minor->eta_inscription
                ;
26         i->moyenne_general = major_minor->moyenne_general
                ;
27         i->modules = major_minor->modules;
28
29         strcpy(major_minor->nom, tmp_nom);
30         strcpy(major_minor->prenom, tmp_prenom);
31         major_minor->numero_dossier = tmp_id;
32         major_minor->eta_inscription =
                tmp_etat_inscription;
33         major_minor->moyenne_general =
                tmp_moyenne_general;
34         major_minor->modules = tmp_modules;
35     }
36     i = i->etudiant_suivant;
37 }

1 void orde_alphabet(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune etudiant\n");
6         return;
7     }

```

```

8
9     Etudiant *i = filiere->premier_du_liste;
10    Etudiant *j;
11
12    char tmp_nom[50];
13    char tmp_prenom[50];
14    int tmp_id;
15    int tmp_etat_inscription;
16    float tmp_moyenne_general;
17    Note *tmp_modules;
18
19    while (i != NULL && i->etudiant_suivant != NULL)
20    {
21        j = i->etudiant_suivant;
22
23        while (j != NULL)
24        {
25            if (strcmp(i->nom, j->nom) > 0)
26            {
27                strcpy(tmp_nom, i->nom);
28                strcpy(tmp_prenom, i->prenom);
29                tmp_id = i->numero_dossier;
30                tmp_etat_inscription = i->eta_inscription;
31                tmp_moyenne_general = i->moyenne_general;
32                tmp_modules = i->modules;
33
34                strcpy(i->nom, j->nom);
35                strcpy(i->prenom, j->prenom);
36                i->numero_dossier = j->numero_dossier;
37                i->eta_inscription = j->eta_inscription;
38                i->moyenne_general = j->moyenne_general;
39                i->modules = j->modules;
40
41                strcpy(j->nom, tmp_nom);
42                strcpy(j->prenom, tmp_prenom);
43                j->numero_dossier = tmp_id;
44                j->eta_inscription = tmp_etat_inscription;
45                j->moyenne_general = tmp_moyenne_general;
46                j->modules = tmp_modules;
47            }
48            j = j->etudiant_suivant; // j avance
49        }
50
51        i = i->etudiant_suivant; // i avance
52    }
53 }

```

3.0.10 Explication de la Fonctions orde_alphabet

Conservant cette fonction elle utilise une trie a bulle, et elle va permettre l'affichage de la liste par ordre alphabetique :

```
1  while (i != NULL && i->etudiant_suivant != NULL)
2  {
3      j = i->etudiant_suivant;
4
5      while (j != NULL)
6      {
7          if (strcmp(i->nom, j->nom) > 0)
8          {
9              strcpy(tmp_nom, i->nom);
10             strcpy(tmp_prenom, i->prenom);
11             tmp_id = i->numero_dossier;
12             tmp_etat_inscription = i->etat_inscription;
13             tmp_moyenne_general = i->moyenne_general;
14             tmp_modules = i->modules;
15
16             strcpy(i->nom, j->nom);
17             strcpy(i->prenom, j->prenom);
18             i->numero_dossier = j->numero_dossier;
19             i->etat_inscription = j->etat_inscription;
20             i->moyenne_general = j->moyenne_general;
21             i->modules = j->modules;
22
23             strcpy(j->nom, tmp_nom);
24             strcpy(j->prenom, tmp_prenom);
25             j->numero_dossier = tmp_id;
26             j->etat_inscription = tmp_etat_inscription;
27             j->moyenne_general = tmp_moyenne_general;
28             j->modules = tmp_modules;
29         }
30         j = j->etudiant_suivant; // j avance
31     }
32
33     i = i->etudiant_suivant; // i avance
34 }
```

4 Modélisation du corps de notre projet : (int main)

C'est ici que nous faisons les appels des différentes fonctions pour donner vie à notre programme.

```
1 int main()
```

```

2 {
3     int votre_choix1 = 1;
4     int votre_choix2 = 1;
5     Etudiant *nouveau_etudiant = NULL;
6     Filiere ma_filiere;
7     ma_filiere.premier_du_liste = NULL;
8     ma_filiere.dernier_du_liste = NULL;
9     ma_filiere.nb_etudiants = 0;
10
11     printf("====BIENVENUE!====\n");
12     printf("Veuillez donner le nom de votre filiere:\n");
13     scanf("%s", ma_filiere.nom_filiere);
14     while (votre_choix1 != 0)
15     {
16         nouveau_etudiant = inscrire_etudiants(&ma_filiere);
17         printf("Voulez vous ajouter ces notes si oui tape (1) \n\n");
18         scanf("%d", &votre_choix2);
19         if (votre_choix2 == 1)
20         {
21             note_etudiants(nouveau_etudiant);
22         }
23         printf("Voulez vous inscrire un nouveau etudiant si oui tape (1) si non (0)\n");
24         scanf("%d", &votre_choix1);
25     }
26     liste_de_la_classe(&ma_filiere);
27     bulletin(&ma_filiere);
28     recherche_et_modification_etudiant(&ma_filiere);
29     supprimer_etudiant(&ma_filiere);
30     maximum(&ma_filiere);
31     minimum(&ma_filiere);
32     du_majorant_minorant(&ma_filiere);
33     printf("-----Classe trie par ordre de merite du\n\n");
34     liste_de_la_classe(&ma_filiere);
35     orde_alphabet(&ma_filiere);
36     printf("-----Classe trie par ordre alphabetique\n\n");
37     liste_de_la_classe(&ma_filiere);
38
39     return 0;
40 }

```